

DM575

Forelæsning 00: Generel information

Casper Bach
Institut for Matematik og Datalogi



Velkommen på andet semester!

Velkommen på andet semester!

- På dette semester lærer I om...
 - Objektorienteret Programmering
 - Databasesystemer
 - Introduktion til kunstig intelligens
 - Algoritmer og datastrukturer

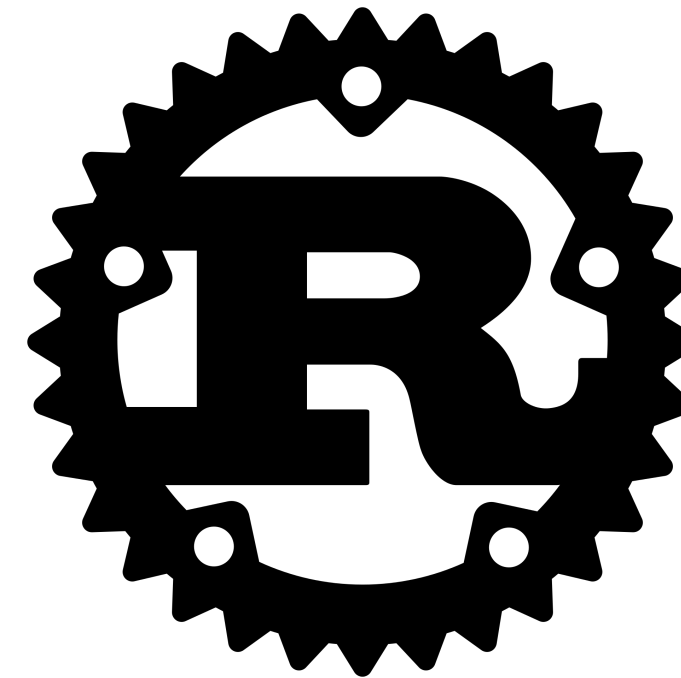
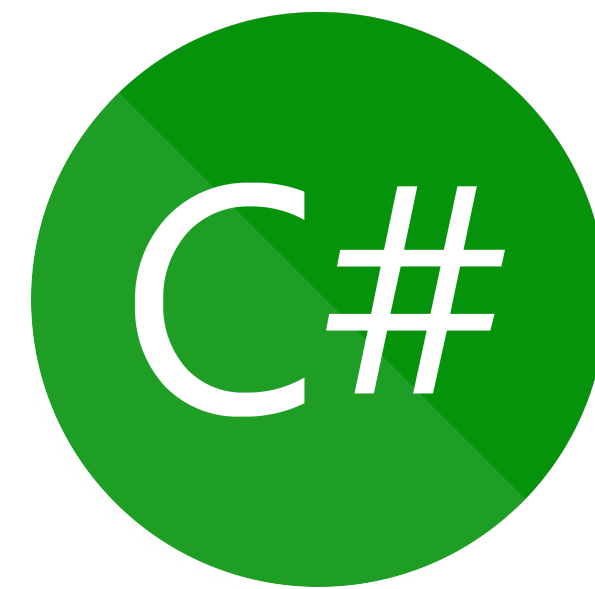
Velkommen på andet semester!

- På dette semester lærer I om...
 - Objektorienteret Programmering
 - Databasesystemer
 - Introduktion til kunstig intelligens
 - Algoritmer og datastrukturer
- Disse er alle *centrale* i både datalogi og kunstig intelligens, så følg godt med.
 - Husk at stille spørgsmål hvis der er noget I ikke forstår!
 - Masser af spørgsmål!

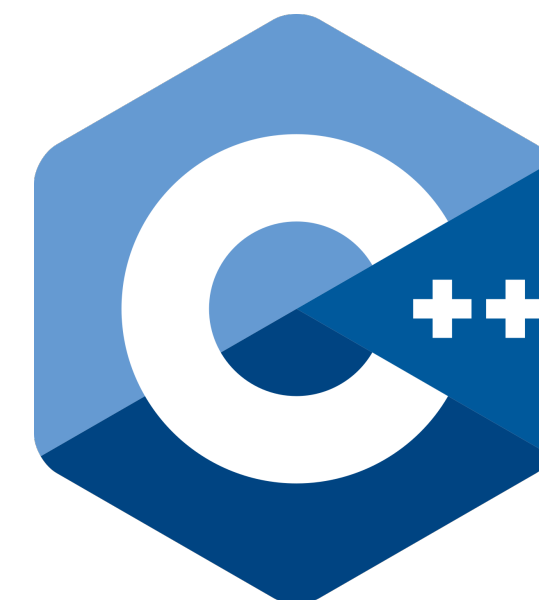
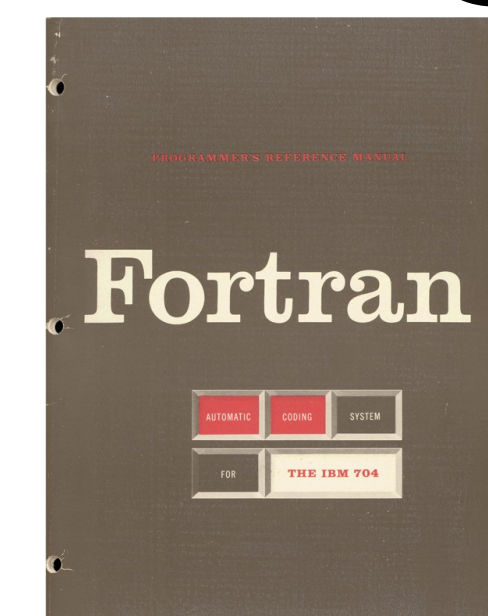
DM574 vs. DM575

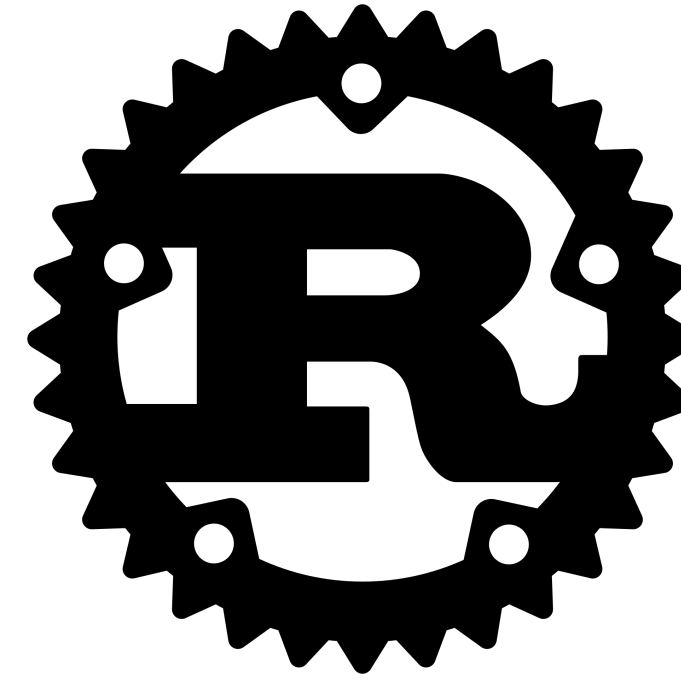
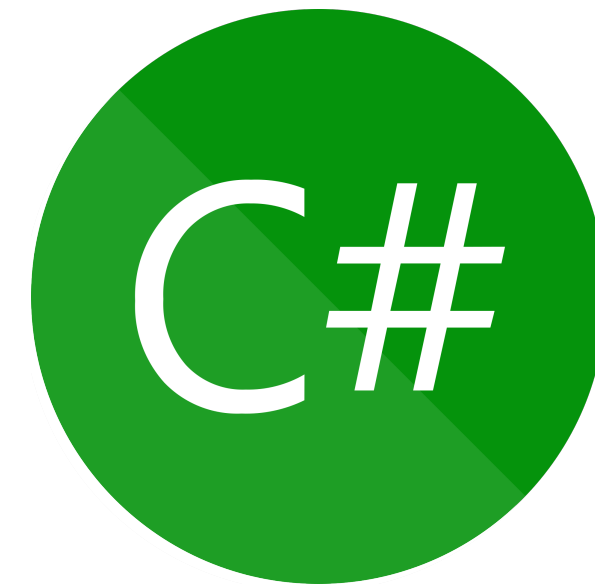
- **DM574: Introduktion til programmering**
 - Fokus på programmers *funktionelle opførsel*.
 - Lille programstørrelse, lav kompleksitet.
 - Python
- **DM575: Objektorienteret programmering**
 - Fokus på programmers *design og struktur*.
 - Principper til design og implementering af softwaresystemer.
 - Stor programstørrelse, høj kompleksitet.
 - Java

**Hvordan designer vi god software
i moderne programmeringssprog?**

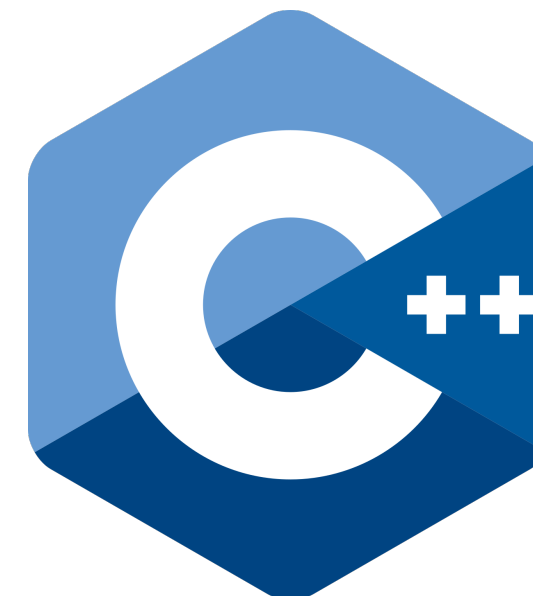
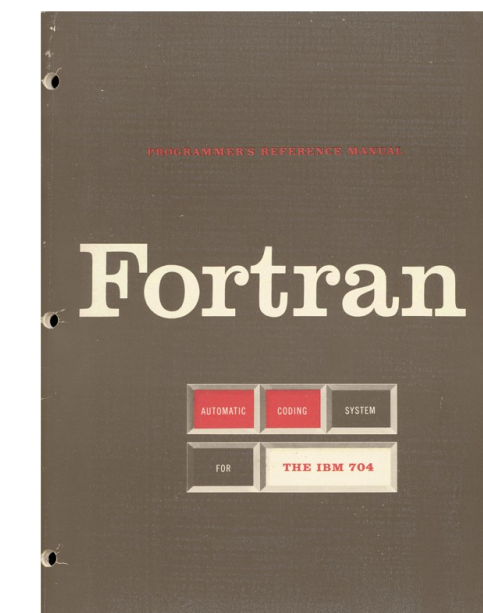


Hvordan designer vi god software i moderne programmeringssprog?





... og hvorfor er det vigtigt?



Developing Software is Hard

Consortium for Information & Software Quality
The Cost of Poor Software Quality in the US: A 2022 Report

Estimated cost of operational software failures in 2022:

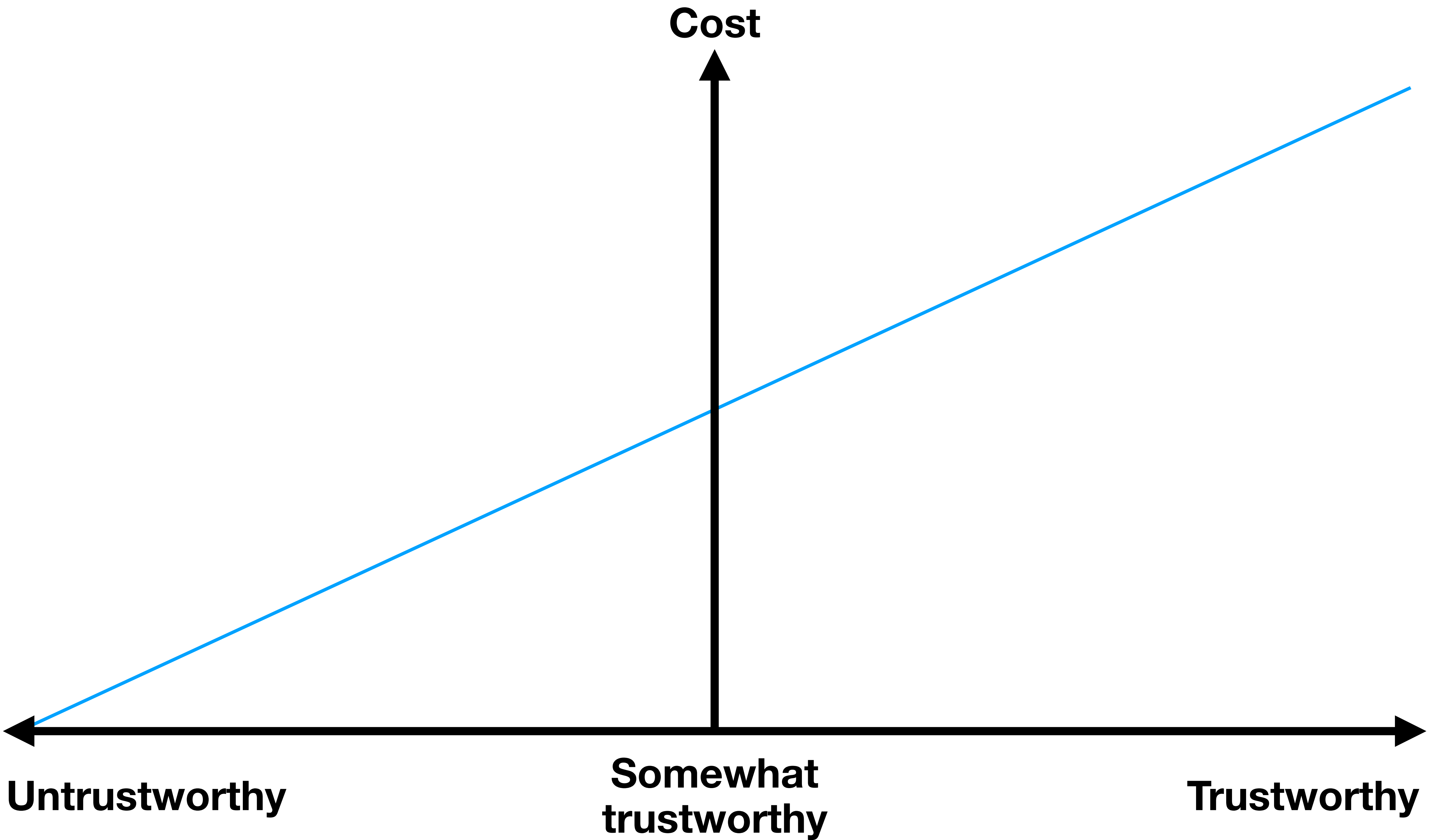
?

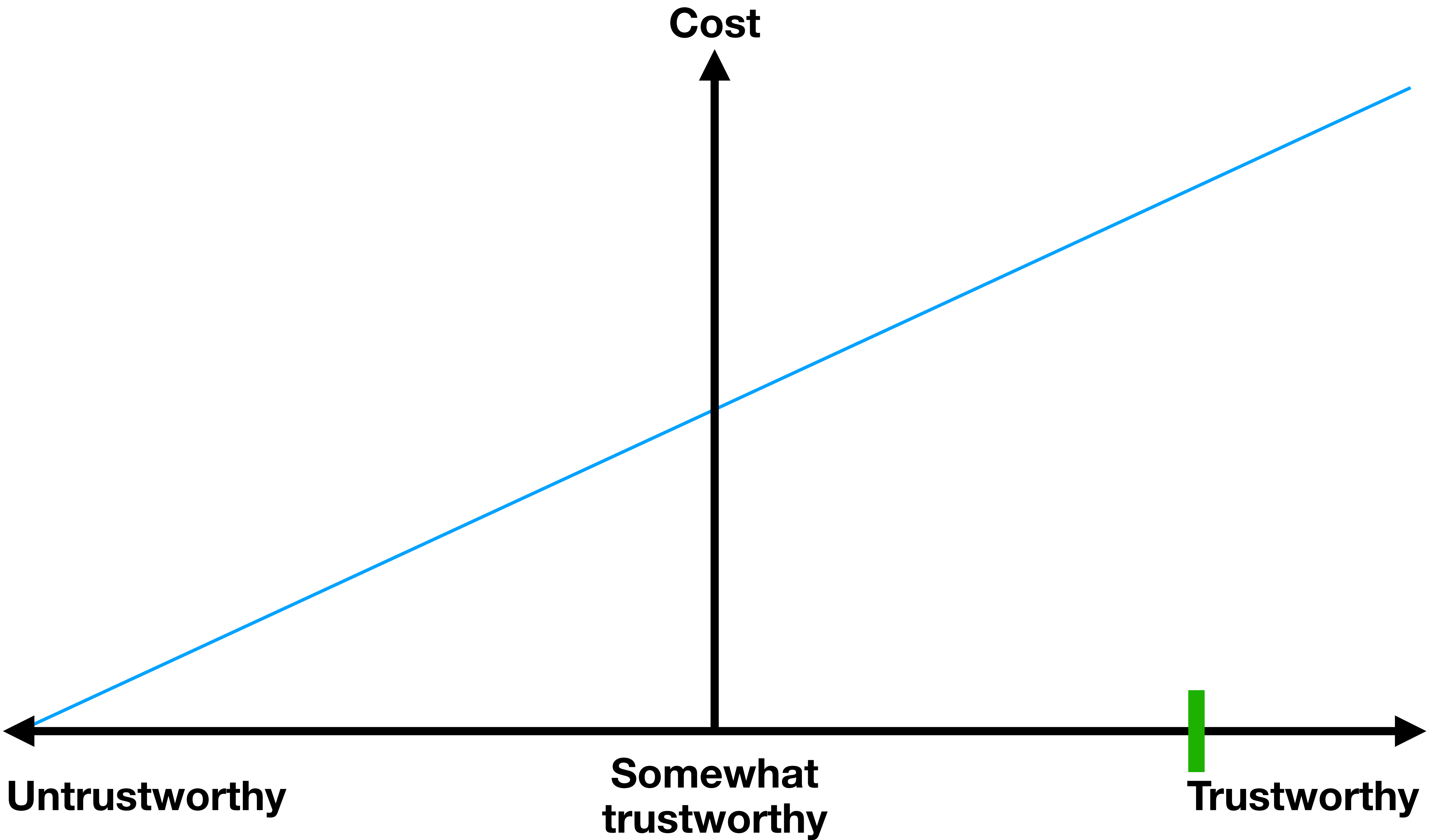
Developing Software is Hard

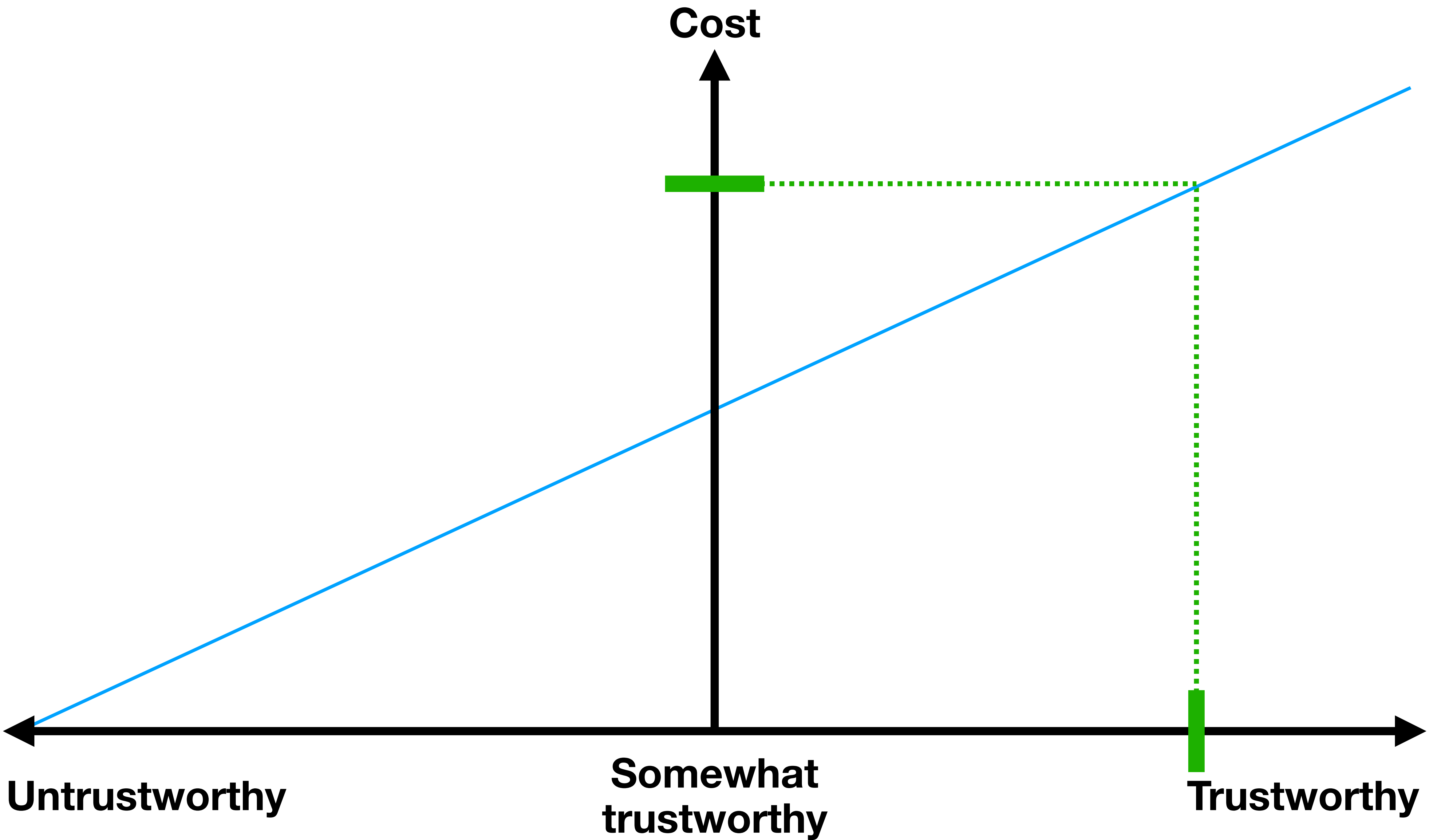
Consortium for Information & Software Quality
The Cost of Poor Software Quality in the US: A 2022 Report

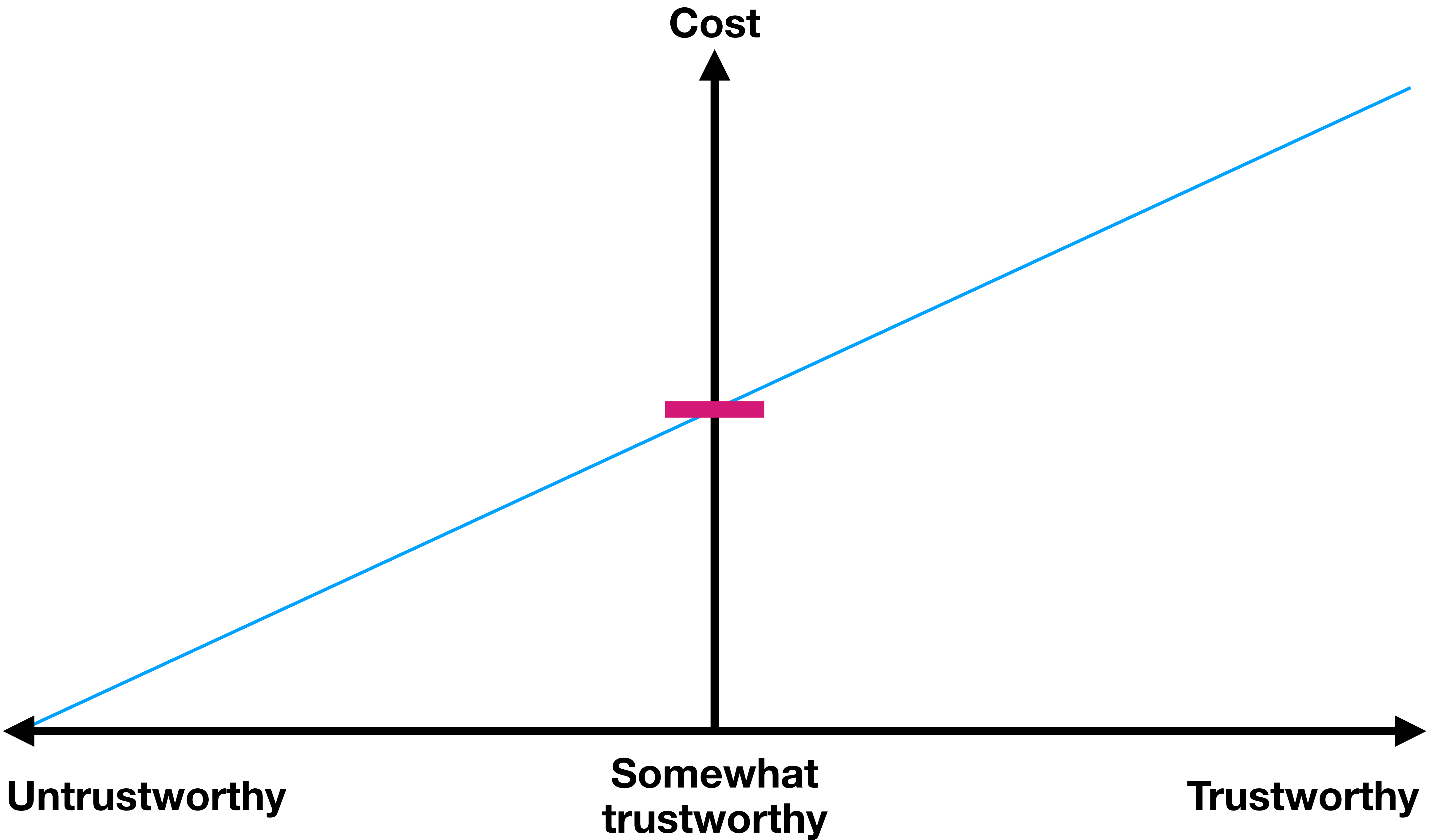
Estimated cost of operational software failures in 2022:

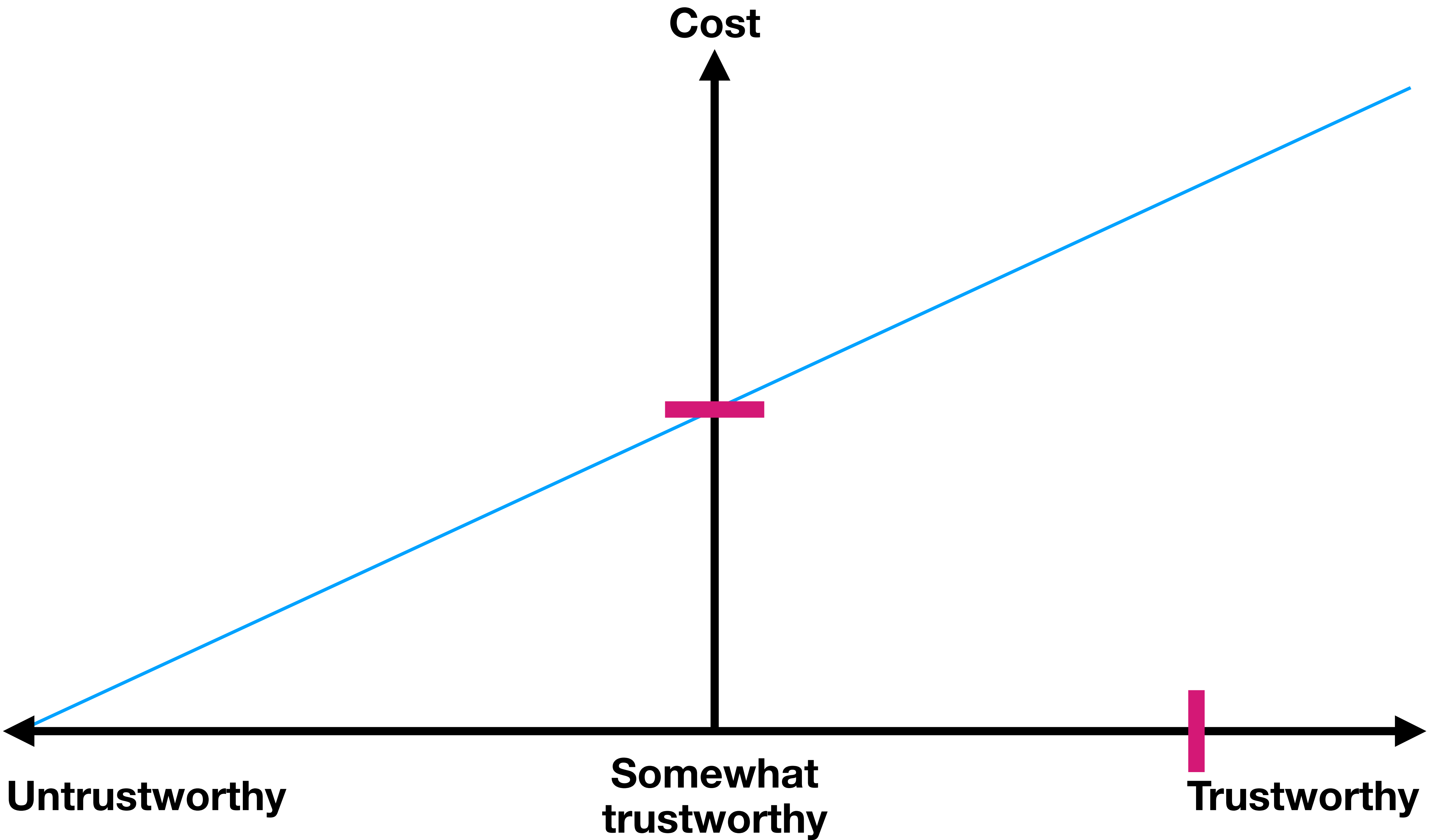
\$ 1,800,000,000,000

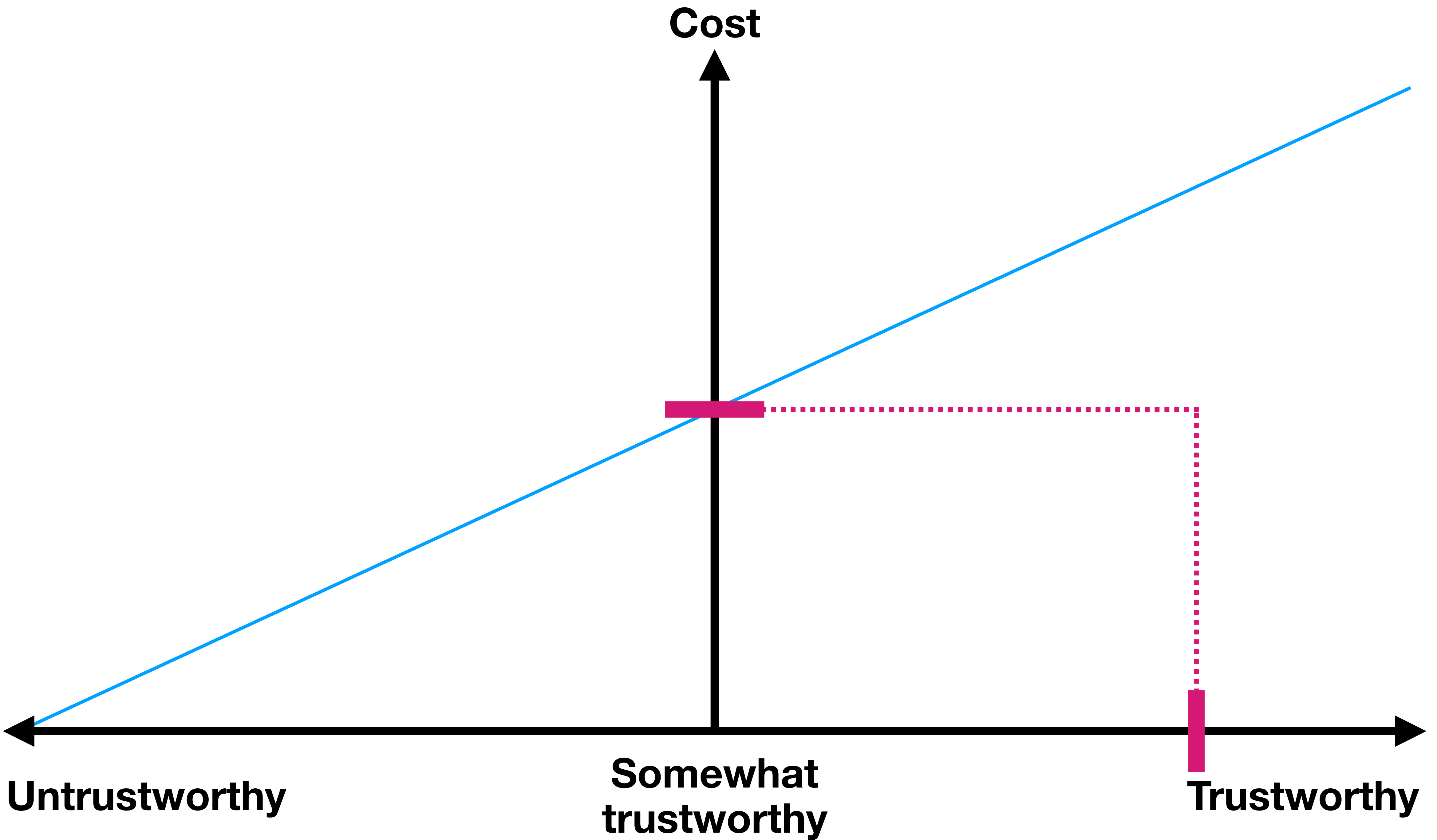


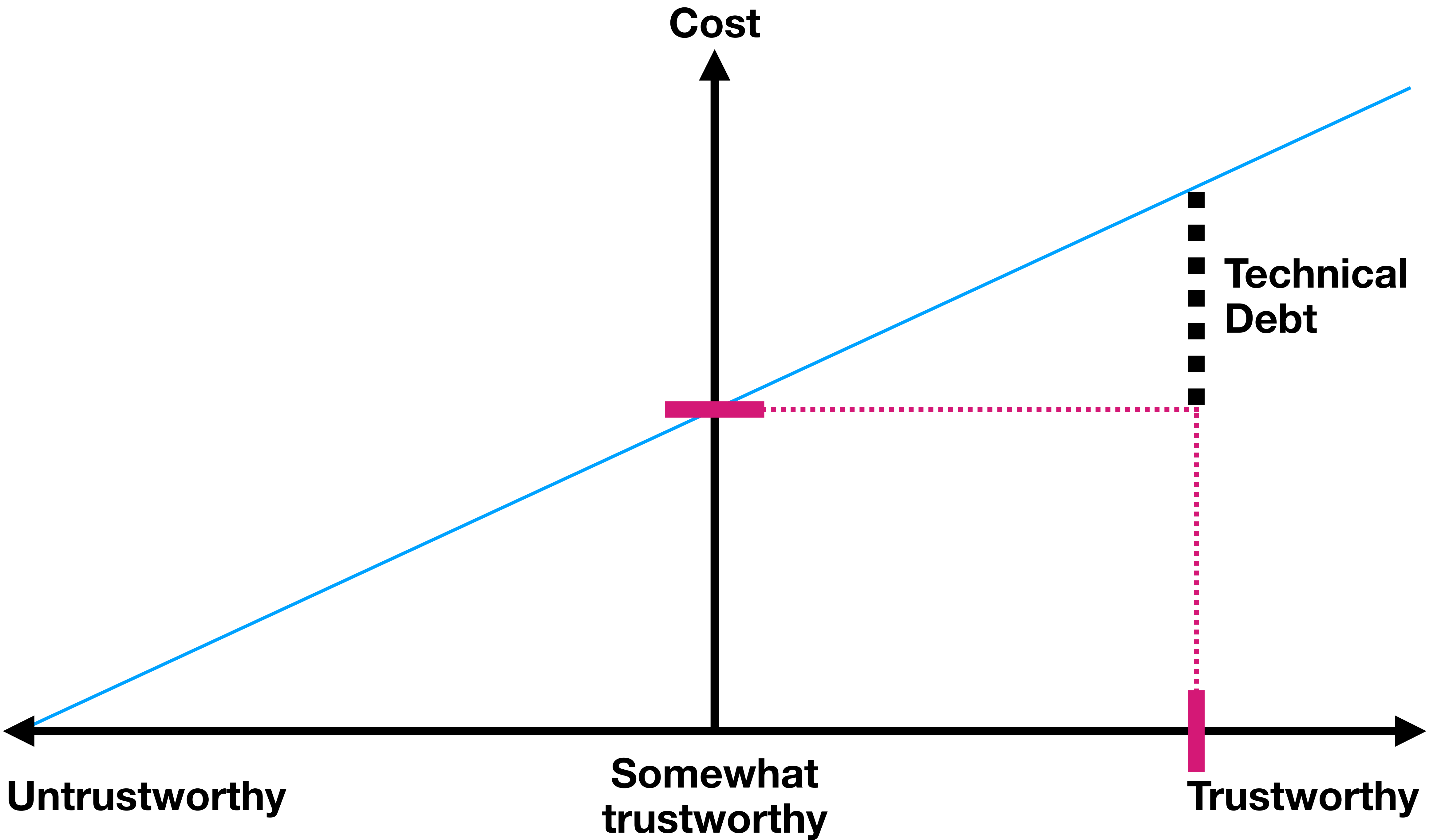












\$1,520,000,000,000

Estimated cost of technical debt in 2022 in the US alone

Programming Languages Helps Write Good Software

- Programming languages come with **tooling** that aids developers to find bugs, write programs, and check requirements
- Programming languages provide **abstractions** that hide irrelevant details of how the underlying machine works, allowing engineers focus on the essential
- Object-oriented programming is an **abstraction for developing abstractions** (e.g., **libraries**)

Hvad er et objekt?

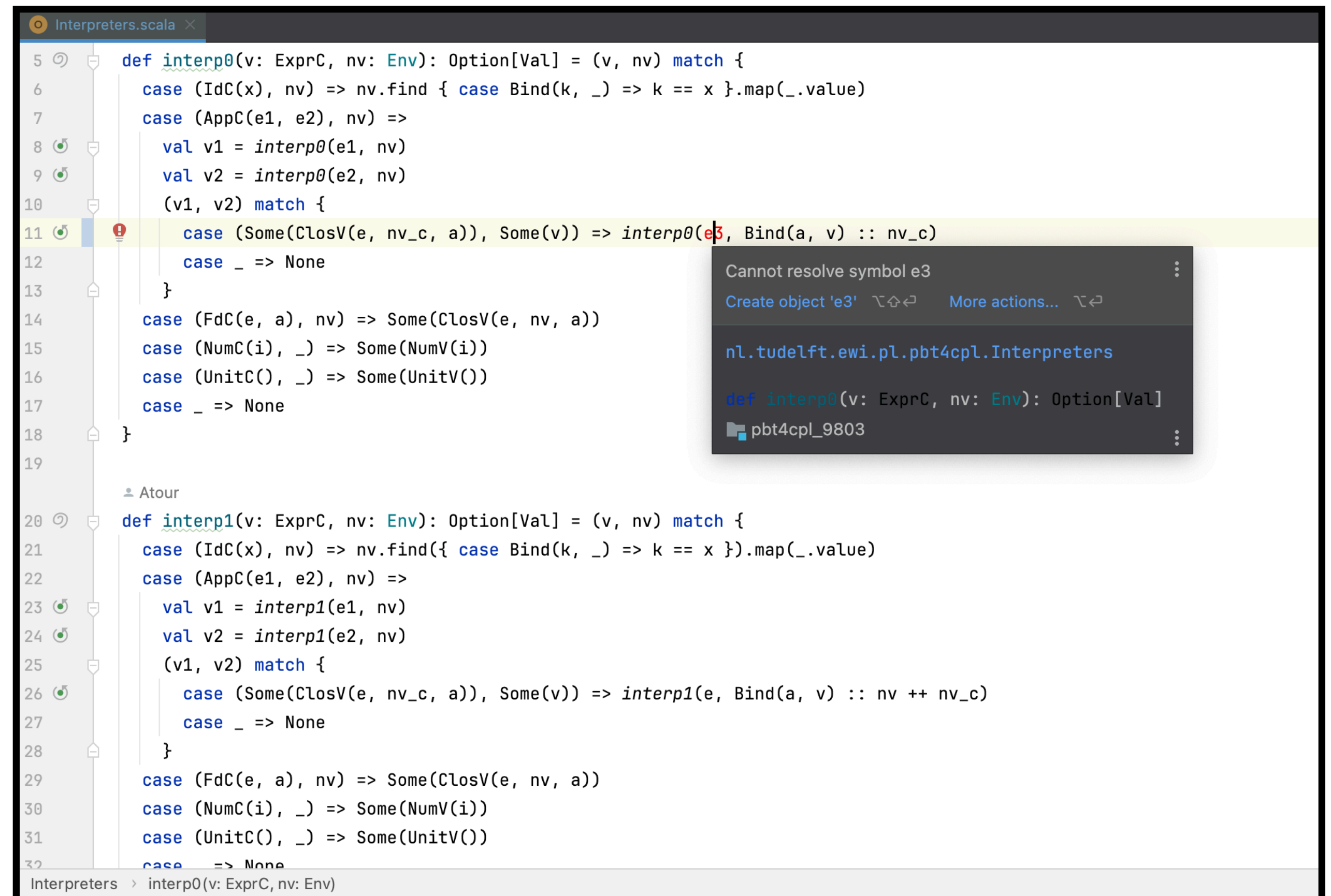
- Et objekt er en mekanisme, der indkapsler (ofte) tilstandsfuld data, samt adfærd der benytter dette.
- Tilstandsfuld data kaldes *felter*. Adfærd kaldes *metoder*.
- En samling af relaterede objekter kaldes ofte et *bibliotek* (eller *library*).
- Gode biblioteker minder om domæne-specifikke programmeringssprog.
- Programmer der bruger sådanne biblioteker er ofte *koncise, nemmere at vedligeholde, effektivt eksekverbare, samt korrekte*.

Programming Language Tooling

Programming Language Tooling

Type Checking

- Type checkers help catch and rule out bugs
- Cheap and easy way to guarantee that (simple) specifications are met
- Typically an integrated part of programming language compilers



The screenshot shows a Scala IDE with two files: `Interpreters.scala` and `Atour`. In `Interpreters.scala`, the `interp0` function is defined with a `match` expression. Line 11 contains a case for `Some(ClosV(e, nv_c, a))` where `interp0(e3, Bind(a, v) :: nv_c)` is called. A red squiggly line under `e3` indicates a type error. A tooltip on the right shows the error message: "Cannot resolve symbol e3". Below the message are links to "Create object 'e3'" and "More actions...". The tooltip also shows the package `nl.tudelft.ewi.pl.pbt4cpl.Interpreters` and the definition of `interp0`. In the `Atour` file, the `interp1` function is defined similarly, but it correctly uses `e` instead of `e3` in its `match` expression.

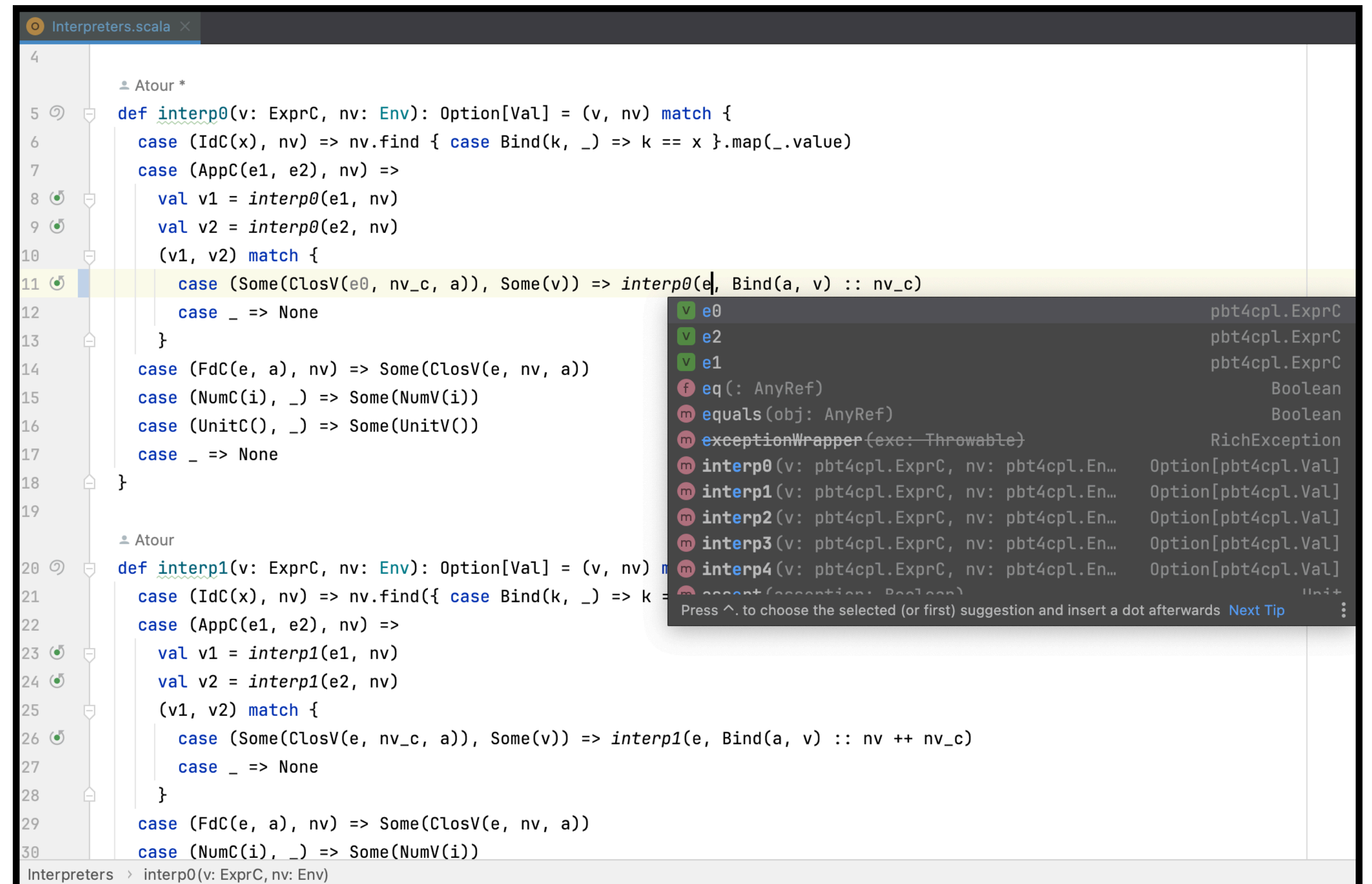
```
def interp0(v: ExprC, nv: Env): Option[Val] = (v, nv) match {
  case (IdC(x), nv) => nv.find { case Bind(k, _) => k == x }.map(_.value)
  case (AppC(e1, e2), nv) =>
    val v1 = interp0(e1, nv)
    val v2 = interp0(e2, nv)
    (v1, v2) match {
      case (Some(ClosV(e, nv_c, a)), Some(v)) => interp0(e3, Bind(a, v) :: nv_c)
      case _ => None
    }
  case (FdC(e, a), nv) => Some(ClosV(e, nv, a))
  case (NumC(i), _) => Some(NumV(i))
  case (UnitC(), _) => Some(UnitV())
  case _ => None
}

def interp1(v: ExprC, nv: Env): Option[Val] = (v, nv) match {
  case (IdC(x), nv) => nv.find({ case Bind(k, _) => k == x }).map(_.value)
  case (AppC(e1, e2), nv) =>
    val v1 = interp1(e1, nv)
    val v2 = interp1(e2, nv)
    (v1, v2) match {
      case (Some(ClosV(e, nv_c, a)), Some(v)) => interp1(e, Bind(a, v) :: nv ++ nv_c)
      case _ => None
    }
  case (FdC(e, a), nv) => Some(ClosV(e, nv, a))
  case (NumC(i), _) => Some(NumV(i))
  case (UnitC(), _) => Some(UnitV())
  case _ => None
}
```

Programming Language Tooling

IDE Auto-Completion

- Auto-completion makes suggestions for how to complete a program
- Makes implementation easier
- Often based on type information to make good suggestions



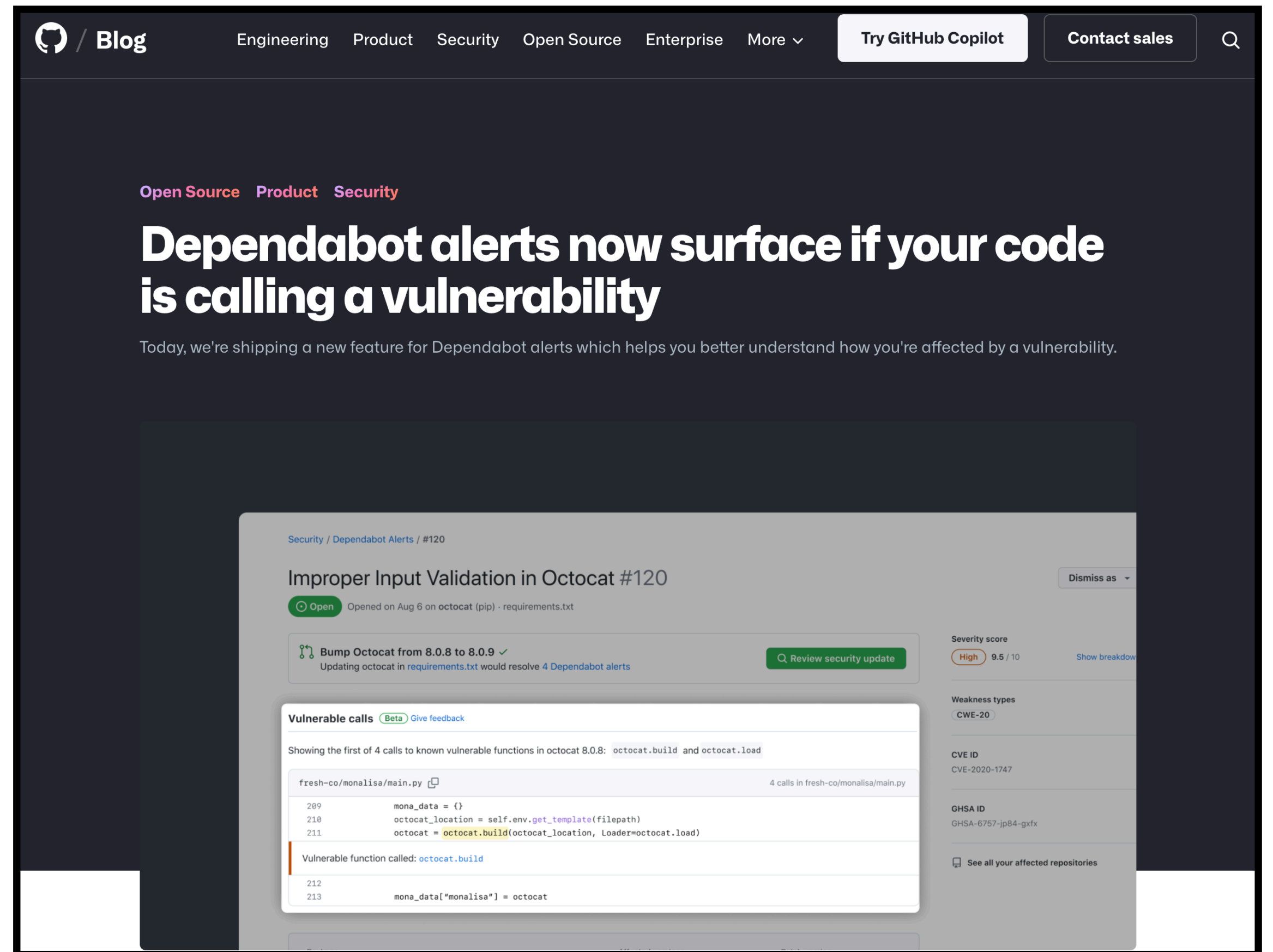
The screenshot shows a code editor with a Scala file named `Interpreters.scala`. The code defines two functions, `interp0` and `interp1`, which take an expression and an environment as input and return an optional value. The `interp0` function is currently being edited, and an auto-completion popup is visible, suggesting various expressions and functions that can be used to complete the line. The popup lists several suggestions, including `e0`, `e2`, `e1`, `eq`, `equals`, `exceptionWrapper`, `interp0`, `interp1`, `interp2`, `interp3`, and `interp4`. The suggestions are color-coded and include their respective types. The code in the background is as follows:

```
4
5 def interp0(v: ExprC, nv: Env): Option[Val] = (v, nv) match {
6   case (IdC(x), nv) => nv.find { case Bind(k, _) => k == x }.map(_.value)
7   case (AppC(e1, e2), nv) =>
8     val v1 = interp0(e1, nv)
9     val v2 = interp0(e2, nv)
10    (v1, v2) match {
11      case (Some(ClosV(e0, nv_c, a)), Some(v)) => interp0(e, Bind(a, v) :: nv_c)
12      case _ => None
13    }
14   case (FdC(e, a), nv) => Some(ClosV(e, nv, a))
15   case (NumC(i), _) => Some(NumV(i))
16   case (UnitC(), _) => Some(UnitV())
17   case _ => None
18 }
19
20 def interp1(v: ExprC, nv: Env): Option[Val] = (v, nv) match {
21   case (IdC(x), nv) => nv.find({ case Bind(k, _) => k == x }).map(_.value)
22   case (AppC(e1, e2), nv) =>
23     val v1 = interp1(e1, nv)
24     val v2 = interp1(e2, nv)
25     (v1, v2) match {
26       case (Some(ClosV(e, nv_c, a)), Some(v)) => interp1(e, Bind(a, v) :: nv ++ nv_c)
27       case _ => None
28     }
29   case (FdC(e, a), nv) => Some(ClosV(e, nv, a))
30   case (NumC(i), _) => Some(NumV(i))
31 }
```


Programming Language Tooling

Static Analysis — Vulnerability Analysis

- Analyze if code calls library with known security issues
- Example: GitHub's *dependabot*
- Implements a *control-flow analyzer* for a programming language



Hvad skal vi lære i dette kursus?

Hvad skal vi lære i dette kursus?

*Teknikker og metoder
til at designe gode biblioteker*

Hvad skal vi lære i dette kursus?

Målbeskrivelse

- Designe objektorienterede modeller for konkrete problemer.
- Udarbejde et klassehierarki baseret på modellen.
- Beskrive og dokumentere det planlagte klassehierarki ved hjælp af standardformater som UML.
- Implementere det planlagte klassehierarki i det konkret anvendte programmeringssprog.
- Planlægge og gennemføre en systematisk aftestning af klassehierarki og objektorienteret program (enhed, komponent, integrationstest).

Kursusplan

- **Programmering i Java** (2 uger)
 - Praktisk introduktion til programmering i et nyt sprog.
 - Emner: Kontrolstrukturer, rækker (arrays), og klasser (classes).
- **Objektorienteret design og implementering** (2 uger)
 - Teori om design og implementering af store systemer.
 - Emner: Nedarvning (inheritance), undtagelser (exceptions), polymorfi (ad-hoc polymorphism), generiske typer (generics/parametric polymorphism), mm.
- **Designmønstre** (2 uger)
 - Standardløsninger på designproblemer.
 - Emner: design patterns, såsom iterator, factory, builder, singleton, adapter, decorator, mm.
- **Projektarbejde** (7 uger)
 - Jeres mulighed for at prøve det hele af i en gruppe på 2-3 studerende.

Kursusmateriale

- **Lærebog**
 - *Java Software Solutions (9th edition)*. J. Lewis and W. Loftus. Pearson, 2018.
- **Noter og øvelser**
 - Egne noter.
 - Øvelsesopgaver.
- **Supplerende materiale (valgfrit)**
 - *Java Design Patterns (3rd edition)*. V. Sarcar. Apress, 2020.
 - *Design Patterns: Elements of Reusable Object-Oriented Software*. E. Gamma, J. Vlissides, R. Johnson, and R. Helm. Addison-Wesley, 1994.



Evaluering

- **Projekt** stilles i ugen 7.-11. april, afleveres 30. maj.*
 - **Reviews:** Præsentation af jeres fremskridt til medstuderende i ugen 21.–25. april og ugen 5.–9. maj.
 - Bygger på materiale fra andre kurser (DM577, DM578) men fokus er på objektorienteret design og programmering.
- **Mundtlig eksamen** i grupper, baseret på jeres indleverede projekt.
 - I den sædvanlige eksamensperiode.
 - Forvent:
 - Praktiske spørgsmål omkr. projekt.
 - Teoretiske spørgsmål vedr. materiale gennemgået til forelæsninger+labøvelser.

* Datoændringer kan forekomme

Undervisere

- **Casper Bach** (kursusansvarlige)
 - casperbach@imada.sdu.dk
 - Kontor: Ø13-604a-1
- **Niels Erik Jepsen** (instruktor, H8, H9)
 - nijep22@student.sdu.dk
- **Sofus Hesseldahl Laubel** (instruktor, H211)
 - slaub21@student.sdu.dk

Undervisere

- **Casper Bach** (kursusansvarlige)
 - I er velkomne til at komme forbi mit kontor uden aftale – men med risiko for, at jeg ikke er der eller er optaget.
 - Send mig en email (oftest nemmere, start emnefeltet med “[DM575]”).
 - Undgå at skrive til mig på itsLearning (jeg får ikke notifikationer).
- **Niels Erik Jepsen** (instruktor) og **Sofus Hesseldahl Laubel** (instruktor)
 - Aftal selv kontaktformer.

Undervisningsformer

- **Forelæsninger**
 - 2x45 minutter hver uge i undervisningsperioden.
 - Introducerer nyt materiale, enten teoretisk eller praktisk.
- **Laboratorietimer (labs)**
 - Erstatte nogle øvelsestimer.
 - Selvstyret arbejde med praktiske øvelser, individuelt eller i grupper, med adgang til instruktør.
 - *Medbring laptop.*

Struktur

- **Undervisningsperioden (3. februar – 14. april)**
 - Ugentlig forelæsning (2x45 min) og øvelsestimer/laboratorietimer (2x45 minutter).
 - Læsning og selvstændigt arbejde med materiale (øvelser mv.)
- **Projektperioden (ca. 14. april – 30. maj; datoændringer kan forekomme)**
 - Selvorganiseret gruppearbejde.
 - Læsning og selvstændigt arbejde med materiale efter behov.
- **Arbejdsbyrde**
 - 7,5 ECTS over et fuldt semester
 - Svarer til en *gennemsnitlig* arbejdsbyrde på 10-12 timer om ugen.

Værktøjer til Java-udvikling

- Papir og blyant, tavle og kridt, etc.
- Simpelt tekstredigeringsværktøj (til at starte med):
 - Microsoft Visual Studio Code (gratis, <https://code.visualstudio.com/>)
 - Udvidelsen “Language Support for Java” af Red Hat.
 - VS Code vil foreslå “Extension Pack for Java,” der har mange avancerede funktioner som vi ikke får brug for til at starte med.
- Kommandolinjen (Terminal, Command Prompt, etc.)
- Java Software Development Kit (SDK), version 8 eller højere.
 - Benyt gerne OpenJDK (<https://adoptium.net/>).
- Sørg for, at disse er installeret på jeres maskiner før første øvelsestid.